

LAPORAN PRAKTIKUM KONTROL CERDAS

SCIKIT-LEARN

Nama : Satya Ilham Pratama

NIM : 234308083

Kelas : TKA 6C

Akun Github : <https://github.com/satyailhampratama>

Abstrak

Perkembangan kecerdasan buatan (Artificial Intelligence) telah mendorong penerapan sistem kontrol cerdas berbasis data melalui pendekatan machine learning. Pada praktikum ini dilakukan implementasi algoritma Random Forest menggunakan library Scikit-learn untuk membangun sistem klasifikasi berbasis citra. Proses dimulai dari pengumpulan dataset menggunakan webcam, dilanjutkan dengan ekstraksi fitur menggunakan MediaPipe untuk memperoleh koordinat landmark, kemudian dilakukan pelatihan dan pengujian model menggunakan metode pembagian data training dan testing. Model yang telah dilatih selanjutnya diimplementasikan untuk klasifikasi secara real-time, termasuk deteksi kondisi tangan (buka/tutup) serta identifikasi posisi tubuh (berdiri/duduk). Evaluasi performa dilakukan menggunakan metrik akurasi untuk mengukur tingkat keberhasilan model dalam mengklasifikasikan data. Hasil praktikum menunjukkan bahwa kombinasi MediaPipe dan Random Forest mampu menghasilkan sistem klasifikasi yang cukup akurat dan responsif secara real-time, sehingga dapat menjadi dasar pengembangan sistem kontrol cerdas yang lebih kompleks di bidang otomasi dan teknologi interaktif.

Kata kunci: Machine Learning, Kontrol Cerdas, Random Forest, Scikit-learn, MediaPipe, Klasifikasi Citra, Ekstraksi Fitur, Deteksi Postur, Computer Vision, Real-time System.

A. Pendahuluan

Perkembangan teknologi di bidang kecerdasan buatan (Artificial Intelligence) telah membawa perubahan signifikan dalam sistem kontrol modern. Sistem kontrol konvensional yang sebelumnya berbasis pemodelan matematis kini mulai dikombinasikan dengan pendekatan berbasis data melalui machine learning. Pendekatan ini dikenal sebagai kontrol cerdas (intelligent control), yaitu sistem yang mampu belajar dari data, mengenali pola, serta mengambil keputusan secara adaptif tanpa harus sepenuhnya bergantung pada model matematis eksplisit (Russell & Norvig, 2021). Salah satu metode machine learning yang banyak digunakan dalam klasifikasi dan prediksi adalah algoritma berbasis ensemble, seperti Random Forest. Algoritma ini bekerja dengan membangun sejumlah decision tree dan menggabungkan hasil prediksinya untuk meningkatkan akurasi serta mengurangi overfitting (Breiman, 2001).

Dalam praktiknya, algoritma ini sering dimanfaatkan dalam berbagai aplikasi kontrol cerdas, termasuk pengenalan pola gerakan, klasifikasi sinyal, serta sistem monitoring berbasis visi komputer. Untuk mengimplementasikan algoritma machine learning secara efisien, diperlukan library yang mendukung proses pemodelan, pelatihan, dan evaluasi secara terstruktur. Salah satu library yang populer dan banyak digunakan dalam penelitian maupun pendidikan adalah Scikit-learn. Library ini menyediakan berbagai algoritma supervised dan unsupervised learning, serta tools untuk preprocessing data, validasi model, dan evaluasi performa (Pedregosa et al., 2011). Keunggulan Scikit-learn terletak pada kemudahan penggunaan, dokumentasi yang lengkap, serta integrasi yang baik dengan library Python lainnya seperti NumPy dan Matplotlib. Pada praktikum ini, dilakukan implementasi algoritma klasifikasi menggunakan Random Forest untuk mengolah data hasil ekstraksi fitur. Data yang digunakan dibagi menjadi data latih (training set) dan data uji (testing set) guna mengevaluasi performa model menggunakan metrik akurasi. Proses ini bertujuan untuk memahami alur kerja machine learning secara sistematis, mulai dari persiapan data, proses training, evaluasi model, hingga penyimpanan model untuk digunakan kembali dalam sistem kontrol cerdas berbasis real-time.

B. Tujuan

Adapun tujuan dari praktikum Scikit-learn ini adalah sebagai berikut:

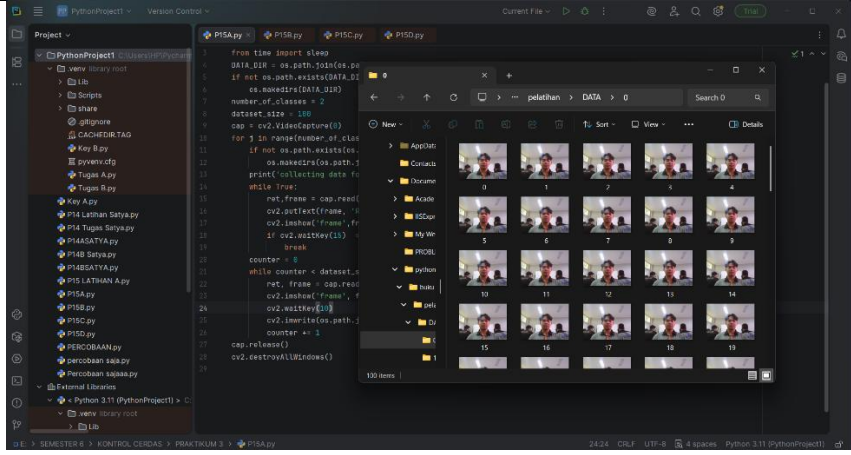
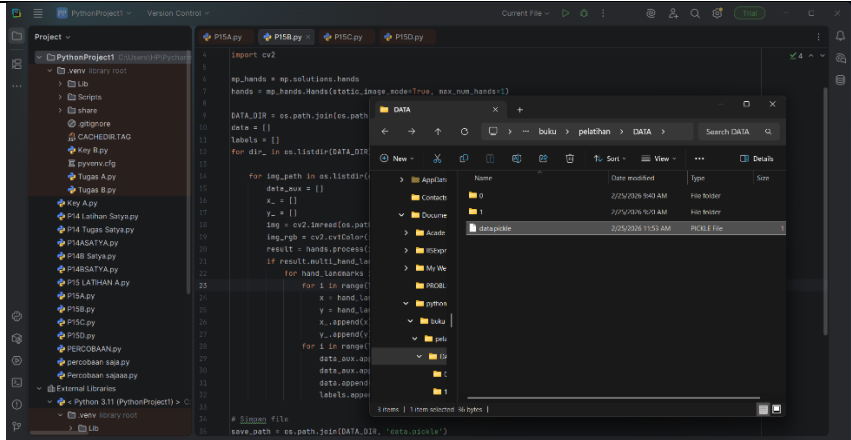
- Memahami konsep dasar machine learning dalam penerapan sistem kontrol cerdas berbasis data.
- Mempelajari proses pengolahan data (data preprocessing) sebelum digunakan dalam pelatihan model.
- Mengimplementasikan algoritma klasifikasi Random Forest menggunakan library Scikit-learn.
- Melakukan pembagian data menjadi data latih (*training set*) dan data uji (*testing set*) untuk mengevaluasi performa model.
- Menghitung dan menganalisis tingkat akurasi model sebagai parameter evaluasi kinerja sistem.

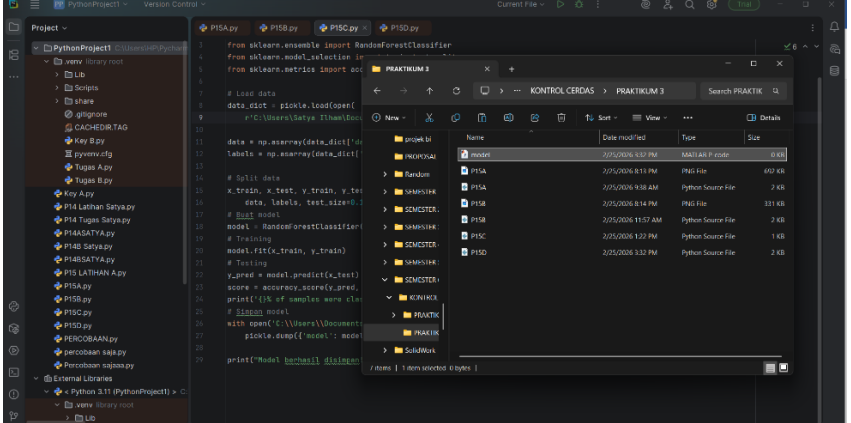
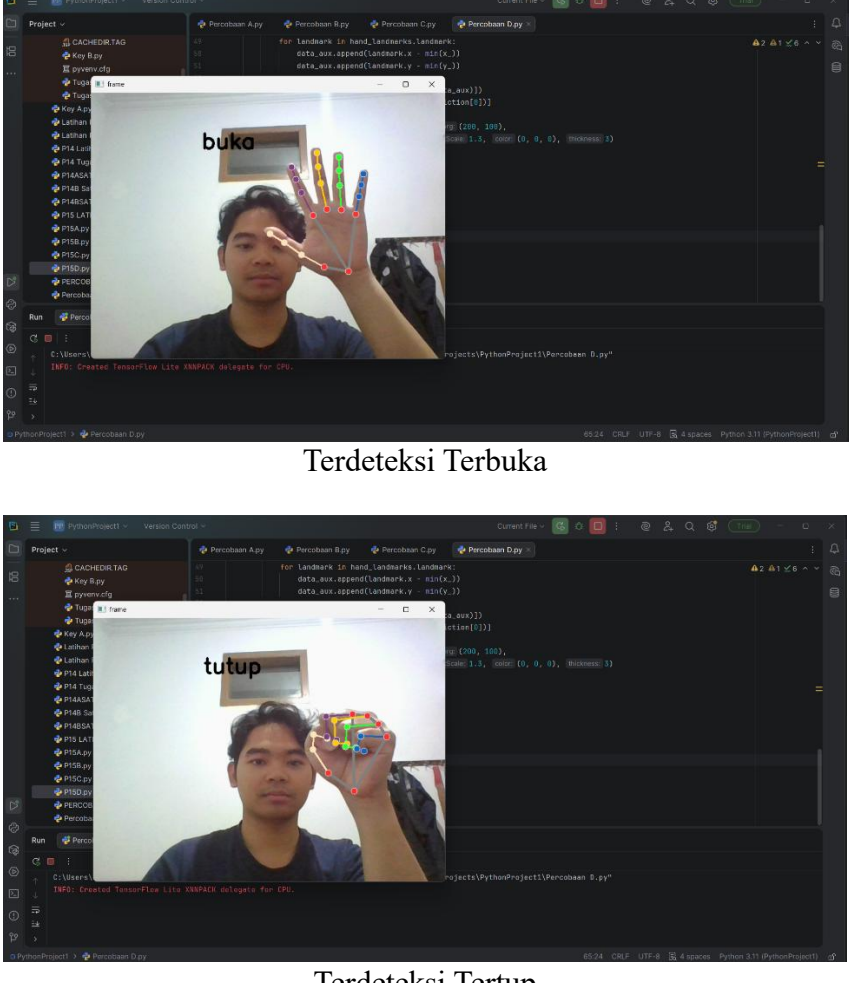
C. Manfaat

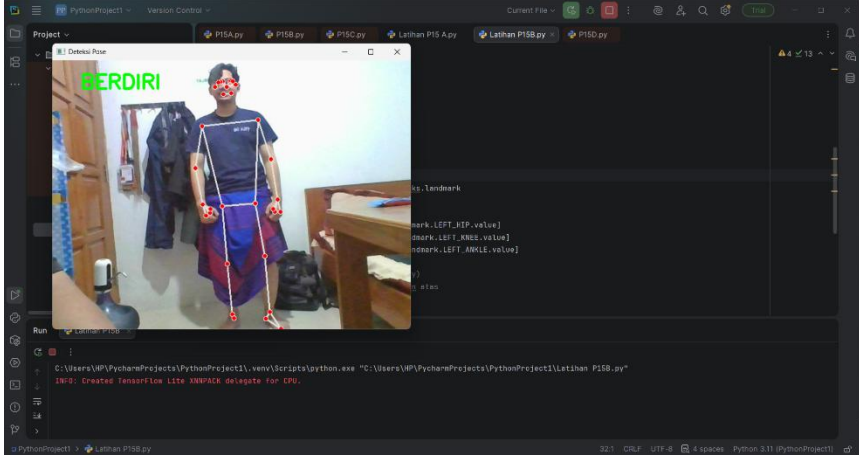
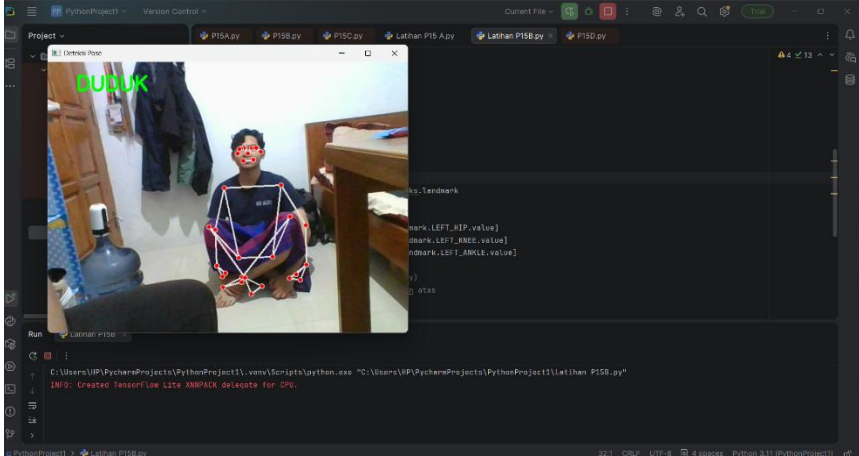
Manfaat yang diperoleh dari praktikum Scikit-learn ini adalah sebagai berikut:

- Mahasiswa memahami alur kerja sistem machine learning secara sistematis, mulai dari input data hingga evaluasi model.
- Meningkatkan kemampuan analisis dalam memilih algoritma yang sesuai untuk permasalahan klasifikasi.
- Memberikan pengalaman langsung dalam implementasi sistem kontrol cerdas berbasis data menggunakan bahasa pemrograman Python.
- Melatih keterampilan pemrograman dan pemecahan masalah dalam pengembangan sistem berbasis kecerdasan buatan.
- Menjadi dasar pengembangan sistem kontrol adaptif dan aplikasi cerdas di bidang teknik, industri, maupun otomasi.

D. Hasil Program

No	Nama Praktikum	Hasil
1.	Pengumpulan Data Menggunakan Webcam	
2.	Ekstraksi Dari Dataset	

3.	Pelatihan Dan Pengujian	 <pre> 1 from sklearn.ensemble import RandomForestClassifier 2 from sklearn.model_selection import train_test_split 3 from sklearn.metrics import accuracy_score 4 5 # Load data 6 data_dict = pickle.load(open('C:\Users\Azy\Documents\data.pkl', 'rb')) 7 8 # Split data 9 data = np.array(data_dict['data']) 10 labels = np.array(data_dict['labels']) 11 12 # Split data 13 X_train, X_test, y_train, y_test = train_test_split(data, labels, test_size=0.2, random_state=42) 14 15 # Build model 16 model = RandomForestClassifier() 17 18 # Training 19 model.fit(X_train, y_train) 20 21 # Testing 22 y_pred = model.predict(X_test) 23 score = accuracy_score(y_test, y_pred) 24 print('The accuracy score is: %f' % score) 25 26 # Save model 27 with open('C:\Users\Azy\Documents\model.pkl', 'wb') as f: 28 pickle.dump(model, f) 29 30 print('Model saved successfully!') </pre>
4.	Implementasi	 Terdeteksi Terbuka Terdeteksi Tertup

No	Nama Praktikum	Hasil
5.	Mendeteksi Seseorang Apakah Berdiri Atau Duduk	 Terdeteksi Berdiri  Terdeteksi Duduk

E. Analisis Program

Program A

Program di atas adalah sebuah skrip Python yang berfungsi untuk secara otomatis mengumpulkan dataset gambar melalui kamera webcam dengan memanfaatkan library OpenCV. Secara keseluruhan, tujuan dari kode ini adalah untuk menghasilkan kumpulan citra (image dataset) yang nantinya dapat digunakan dalam aplikasi machine learning, seperti klasifikasi gambar atau pengenalan objek. Pada bagian awal, program mengimpor library os, cv2, dan sleep dari modul time. Library os digunakan untuk mengelola direktori atau folder tempat penyimpanan data, sedangkan cv2 (OpenCV) memiliki fungsi untuk mengakses kamera serta memproses gambar. Variabel DATA_DIR menyimpan lokasi penyimpanan dataset, yaitu di dalam folder Documents/python/buku/pelatihan/DATA di komputer pengguna. Kemudian, program memeriksa apakah folder tersebut sudah ada. Jika folder tersebut belum ada, maka folder akan dibuat secara

otomatis dengan menggunakan `os.makedirs()`. Selanjutnya, dua parameter utama ditetapkan, yaitu `number_of_classes = 2` yang menunjukkan ada dua kelas data (misalnya kelas 0 dan kelas 1), serta `dataset_size = 100` yang berarti setiap kelas akan memiliki 100 gambar. Kamera diaktifkan dengan menggunakan `cv2.VideoCapture(0)`, di mana angka 0 menunjukkan bahwa itu adalah webcam utama dari perangkat. Program kemudian melakukan iterasi sebanyak jumlah kelas yang sudah ditentukan. Untuk masing-masing kelas, sistem akan menciptakan folder baru sesuai dengan nomor kelas (seperti folder “0” dan “1”) jika folder itu belum ada. Selanjutnya, tampilan kamera akan muncul dengan instruksi teks “Ready? Press ‘Q’! ”. Tahap ini berfungsi sebagai waktu persiapan bagi pengguna sebelum pengambilan gambar dimulai. Ketika tombol “Q” ditekan, pengambilan dataset dimulai. Saat pengambilan data, program akan terus menangkap gambar hingga mencapai 100 gambar sesuai dengan nilai `dataset_size`. Setiap frame yang berhasil ditangkap akan langsung ditampilkan dan disimpan ke dalam folder kelas yang relevan dengan format nama file yang berurutan (0. jpg, 1. jpg, 2. jpg, dan seterusnya). Proses ini berlangsung secara otomatis tanpa memerlukan penekanan tombol tambahan. Setelah semua gambar dari setiap kelas terkumpul, kamera akan dinonaktifkan dengan menggunakan `cap.release()` dan semua jendela tampilan akan ditutup dengan `cv2.destroyAllWindows()`.

Program B

Kode di atas merupakan program Python yang berfungsi untuk melakukan ekstraksi fitur (feature extraction) dari dataset gambar tangan menggunakan MediaPipe Hands, kemudian menyimpannya dalam bentuk file `.pickle` untuk keperluan pelatihan model machine learning. Jika pada kode sebelumnya dilakukan proses pengambilan gambar (dataset collection), maka pada kode ini dilakukan proses pengolahan data agar siap digunakan untuk training. Pada bagian awal, program mengimpor beberapa library penting, yaitu `os` untuk mengelola direktori, `pickle` untuk menyimpan data dalam format biner, `mediapipe` untuk mendeteksi landmark tangan, dan `cv2` (OpenCV) untuk membaca serta memproses gambar. Library MediaPipe dikonfigurasi dengan `static_image_mode=True`, yang berarti sistem akan memproses gambar satu per satu (bukan video real-time), serta `max_num_hands=1` yang membatasi deteksi hanya satu tangan dalam setiap gambar. Program kemudian menentukan lokasi dataset yang sebelumnya telah dibuat. Dua variabel utama disiapkan, yaitu data untuk menyimpan hasil ekstraksi fitur dan labels untuk menyimpan label kelas dari setiap gambar. Struktur dataset diasumsikan berbentuk folder per kelas (misalnya folder “0” dan “1”), sehingga program melakukan perulangan untuk membaca setiap folder kelas dan setiap gambar di dalamnya. Setiap gambar dibaca menggunakan `cv2.imread()`, lalu dikonversi dari format warna BGR ke RGB karena MediaPipe bekerja dalam format RGB. Selanjutnya, gambar diproses menggunakan `hands.process()` untuk mendeteksi landmark tangan. Jika tangan terdeteksi (`result.multi_hand_landmarks` bernilai `True`), maka sistem akan mengambil seluruh koordinat landmark tangan tersebut. MediaPipe Hands sendiri mendeteksi 21 titik landmark pada tangan, dan setiap titik memiliki koordinat `x` dan `y` yang telah dinormalisasi (bernilai antara 0 sampai 1). Koordinat `x` dan `y` dari setiap landmark terlebih dahulu dikumpulkan ke dalam list terpisah (`x_` dan `y_`). Setelah itu dilakukan proses normalisasi posisi dengan cara mengurangi

setiap koordinat terhadap nilai minimum x dan minimum y pada gambar tersebut. Tujuannya adalah agar data menjadi relatif terhadap posisi tangan, sehingga model tidak bergantung pada letak absolut tangan di dalam gambar. Nilai hasil normalisasi ini kemudian dimasukkan ke dalam `data_aux` sebagai fitur numerik. Setelah semua koordinat diproses, data fitur dimasukkan ke dalam list data, sedangkan nama folder (yang merepresentasikan kelas) dimasukkan ke dalam list labels. Dengan demikian, setiap gambar akan memiliki sekumpulan fitur numerik (berasal dari landmark tangan) dan label kelas yang sesuai.

Program C

Kode yang terdapat di atas menjelaskan proses pelatihan dan pengujian model machine learning dengan memanfaatkan algoritma Random Forest untuk mengidentifikasi data gerakan tangan yang telah diambil dan disimpan dalam file data. `pickle`. Proses dimulai dengan memuat data melalui modul `pickle`, kemudian memisahkan isi file menjadi dua bagian utama, yakni data (fitur numerik hasil ekstraksi landmark tangan) dan label (kelas atau kategori gerakan). Data tersebut selanjutnya dikonversi ke dalam format array menggunakan NumPy agar bisa diproses oleh pustaka `scikit-learn`. Setelah itu, dataset dibagi menjadi data latihan dan data pengujian menggunakan fungsi `train_test_split()` dengan komposisi 90% untuk data latihan dan 10% untuk data pengujian. Parameter `stratify=labels` diterapkan agar proporsi setiap kelas tetap konsisten antara data latihan dan data pengujian, sehingga model tidak condong ke salah satu kelas. Selanjutnya, model `RandomForestClassifier` dibuat, yang merupakan algoritma klasifikasi berbasis kumpulan pohon keputusan yang berfungsi dengan metode voting untuk menentukan hasil prediksi. Model kemudian dilatih menggunakan data latihan melalui fungsi `fit()`. Setelah latihan selesai, model diuji dengan data pengujian untuk mengevaluasi performanya. Hasil prediksi dibandingkan dengan label yang sebenarnya menggunakan `accuracy_score()`, sehingga diperoleh persentase akurasi yang mengindikasikan seberapa baik model mampu mengklasifikasikan data dengan akurat. Nilai akurasi ini kemudian ditampilkan dalam format persentase. Langkah terakhir adalah menyimpan model yang telah dilatih ke dalam file `model.p` menggunakan `pickle`, agar model tersebut dapat digunakan kembali tanpa perlu melakukan pelatihan ulang. Secara keseluruhan, kode ini merupakan tahap akhir dari sistem klasifikasi gerakan tangan, yaitu merancang, menguji, dan menyimpan model yang siap digunakan untuk pengolahan prediksi secara real-time.

Program D

Kode di atas dirancang untuk mendeteksi dan mengklasifikasikan posisi tangan secara langsung menggunakan kamera. Pertama, kode ini memuat model pembelajaran mesin dari file `model.pickle` yang sudah dilatih untuk mengenali dua keadaan tangan: "buka" dan "tutup". Webcam kemudian diaktifkan dengan bantuan OpenCV, dan MediaPipe Hands digunakan untuk mendeteksi keberadaan tangan dalam setiap bingkai video. Setiap frame yang diterima diubah ke dalam format RGB, lalu diproses oleh MediaPipe untuk memperoleh landmark tangan, yaitu titik-titik penting pada jari-jari dan telapak tangan. Setelah memperoleh landmark, kode ini menghitung

posisi x dan y dari setiap titik, menormalkan koordinat berdasarkan titik terendah, dan menyimpannya dalam `data_aux` sebagai masukan untuk model. Selanjutnya, model akan memprediksi kondisi tangan, dan hasil dari prediksi ("buka" atau "tutup") akan ditampilkan secara langsung di layar menggunakan `cv2. putText`. Seluruh proses ini dilaksanakan secara terus-menerus dalam sebuah loop hingga pengguna menekan tombol 'q' untuk keluar. Secara keseluruhan, kode ini mengintegrasikan pengolahan citra secara real-time, ekstraksi fitur landmark tangan, serta klasifikasi dengan model pembelajaran mesin untuk menciptakan interaksi visual antara tangan pengguna dan sistem.

Program Latihan

Program ini merupakan penerapan untuk mendeteksi posisi tubuh dengan menggunakan MediaPipe Pose dan OpenCV, bertujuan untuk mengetahui apakah seseorang sedang berdiri atau duduk secara langsung melalui webcam. Di awal program, modul `mediapipe. pose` diinisialisasi untuk menemukan titik-titik landmark pada tubuh, dan `drawing_utils` digunakan untuk menggambar kerangka pada setiap frame video. Untuk menangkap input secara langsung, kamera diaktifkan dengan `cv2. VideoCapture(0)`. Dalam siklus utama, setiap frame yang diambil akan dibalik secara horizontal dengan `cv2. flip(frame, 1)` agar hasilnya tampak seperti di cermin, membuatnya lebih alami saat digunakan. Setelah itu, frame yang diambil diubah dari format BGR menjadi RGB karena MediaPipe hanya dapat mengolah gambar dalam format RGB. Proses yang dihasilkan akan disimpan dalam variabel `results`. Ketika landmark tubuh berhasil terdeteksi (jika `results. pose_landmarks` tidak kosong), program akan menggambar titik-titik dan garis yang membentuk kerangka tubuh pada frame tersebut. Kemudian, tiga titik penting yang diambil adalah pinggul, lutut, dan pergelangan kaki bagian kiri. Namun, untuk proses klasifikasi, hanya perbandingan koordinat vertikal (nilai y) antara pinggul dan lutut yang digunakan. Dalam sistem koordinat MediaPipe, semakin besar nilai y berarti posisi semakin ke bawah. Maka jika `knee.y` lebih besar dari `hip.y` (lutut berada di bawah pinggul), program mengidentifikasi posisi sebagai "BERDIRI". Sebaliknya, jika kondisi tersebut tidak terjadi, maka dianggap sebagai "DUDUK". Metode ini termasuk sederhana karena hanya mengandalkan perbandingan posisi vertikal tanpa menghitung sudut lutut atau mempertimbangkan kedua kaki. Pendekatan ini cukup efektif dalam kondisi kamera yang tegak dan tubuh terlihat dengan jelas, namun mungkin kurang tepat jika sudut pengambilan gambar miring atau posisi duduk tidak terlalu menekuk. Secara keseluruhan, kode ini memperlihatkan dasar-dasar klasifikasi postur tubuh yang berbasis landmark dengan logika yang sederhana, sehingga sangat cocok sebagai langkah awal dalam memahami deteksi pose menggunakan MediaPipe.

F. Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. Implementasi algoritma Random Forest menggunakan Scikit-learn dapat digunakan secara efektif untuk sistem klasifikasi berbasis citra dalam kontrol cerdas.
2. Proses machine learning yang sistematis, mulai dari pengumpulan data, ekstraksi fitur, pelatihan, hingga evaluasi model, sangat berpengaruh terhadap performa sistem.
3. Penggunaan MediaPipe dalam ekstraksi landmark mempermudah proses pengolahan fitur karena menghasilkan data numerik yang terstruktur dan konsisten.
4. Model yang telah dilatih mampu melakukan klasifikasi secara real-time dengan tingkat akurasi yang baik.
5. Sistem deteksi postur berdiri dan duduk dapat diimplementasikan menggunakan logika sederhana berbasis koordinat landmark, meskipun masih memiliki keterbatasan pada kondisi sudut pengambilan gambar tertentu.

G. Saran

saran untuk pengembangan lebih lanjut adalah sebagai berikut:

1. Menambahkan jumlah dataset dan variasi kondisi pencahayaan untuk meningkatkan generalisasi model.
2. Menggunakan metode evaluasi tambahan seperti confusion matrix, precision, recall, dan F1-score agar analisis performa model lebih komprehensif.
3. Menguji algoritma lain seperti Support Vector Machine (SVM) atau Neural Network untuk membandingkan performa dengan Random Forest.

Referensi

Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.

Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python.

Journal of Machine Learning Research, 12, 2825–2830.

Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.

Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow* (3rd ed.). O'Reilly Media.